Toan Ly

CS 5137 - Deep Learning

Homework 3: Medical Image Segmentation

Step 1. Data

1) The provided dataset has 2 sets, training set and test set. Each set has 2 separate folders containing the original images and their corresponding ground truth segmentation masks. The training set has 80 images and masks, and the test set has 20 images and masks. Therefore, the overall dataset contains 100 images in total.

2) This dataset addresses retinal vessel segmentation problem. The goal is to classify each pixel in the image as either blood vessel (1), or background (0).

3) All the images in this dataset have the same size as 512 x 512. The fundus images have the shape of 3 x 512 x 512 (in RGB), and their corresponding masks have the shape of 1 x 512 x 512 (in greyscale).

Additionally, the pixel intensity range in both the images and masks are [0, 255]. Therefore, the masks will go through preprocessing to transform into binary mask.

4) This dataset doesn't have any missing information. All the folders have a pair of images and masks.

5) The goal label of this dataset should be either 1 (vessel) or 0 (background). However, the current masks are in the range of [0, 255], transforming them into binary is necessary for training the model.

6) Since the dataset is already split into train and test with the ratio of 8:2, I'll further split the training set into training (60 images) and validation sets (20 images). Therefore, the overall ratio is 6:2:2

7) Because only 60 images are available for training, which is usually not a sufficient number in regular deep learning setup, these training images will go through extensive augmentation steps to expose the model to more variations of the data and avoid overfitting. For augmentation, I utilized MONAI library as it is popular and widely adopted in medical imaging

- **Mask binarization**: For the training augmentation part, since the mask is not in binary format, it is transformed such that any pixel values greater than a threshold (I used threshold of 0 in this case) will be 1 (vessel), and others will be 0 (background)

- **Add small vessels:** To improve vessel representation, in this step, I specifically applied a morphological opening operation on the binary mask to remove the small vessels. Then, by subtracting the opened mask from the original mask, I obtained a mask highlighting only the small and thin vessels. These small vessel regions were then merged back with the main vessels to form a new mask that preserves both large and small vessels. The intuition behind this approach is to encourage the model to pay more attention to the small vessels during training, effectively weighting the smaller vessels more strongly in the learning process. As a result, the goal of segmentation now becomes a 3-class problem (background, regular vessels, and thin vessels) instead of just 2 (background and vessels)

- **CLAHE**: During EDA, I observed that the green channel seems to exhibit a better contrast among the all the channels. Therefore, I applied CLAHE (Contrast Limited Adaptive Histogram Equalization) technique on the green channel to enhance the contrast further, making small vessels more visible, and append this as an additional channel to the input image. Therefore, the input now has the shape of 4 x 512 x 512.

- **Normalization**: these training image intensities will be rescaled to [0, 1]

- **Geometric augmentation**: random horizontal, vertical flips, rotations, and small grid distortions

- **Photometric augmentation**: random contrast adjustment, intensity scaling, and Gaussian noise injection

Finally, the images are randomly cropped in 128 x 128 patches with the intuition of increasing the number of training samples and that the model can focus more on smaller regions with fine vessel details. In the setup, the images are randomly cropped into 8 patches. Although these 8 patches don't cover the whole original image, if the model is trained long enough, it will have sampled the whole image multiple times over many epochs.

For validation and test sets, the images will only go through basic preprocessing steps such as adding additional small vessel class, appending CLAHE-applied green channel, and normalizing to [0, 1]

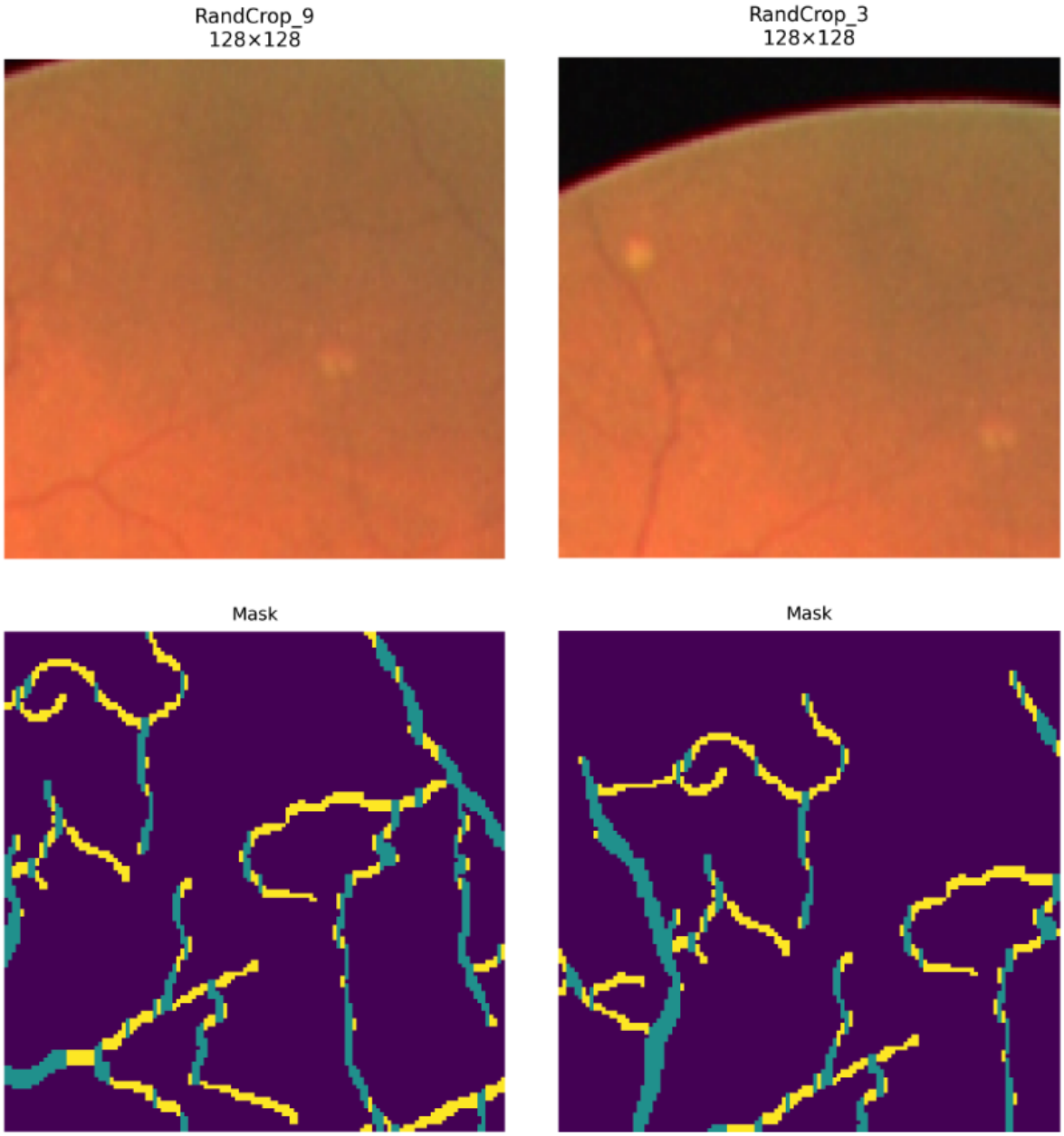


Figure 1. Training images after preprocessing. Yellow represents the small vessel class

Step 2. Model

For this task, I implemented and trained UNet from scratch with PyTorch, experimenting with different depth levels and variations to evaluate their impact on segmentation performance.

1. **UNet** (2, 3, and 4 levels of depth)
2. **Residual UNet (ResUNet)** (3 and 4 levels of depth):
   This replaces the standard convolutional blocks with residual blocks

For hyperparameters, I used different training batch size as long as it fits the 15GB GPU of Google Colab. Additionally, I used Leaky ReLU activation, batch normalization

The initial number of filters was set to 64 and increased as the network depth expanded (64 -> 128 -> 256 -> 512).

For upsampling methods, I tested transpose convolution, upsampling bilinear, and nearest. Among these, nearest yielded the best results for UNet in terms of Dice and IoU. However, ResUNet performed better with transpose convolution.

Step 3: Objective

For loss function, I used the weighted combination of Dice Loss and Cross Entropy Loss.

- Dice Loss focuses on region-level similarity between the prediction and ground truth. It is sensitive to overlap
- Cross Entropy Loss (CE) focuses on pixel level classification accuracy

Through experiments, I observed that this hybrid loss yielded much better results compared to training with only one loss (Dice or CE).

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{Dice} + \beta \cdot \mathcal{L}_{CE}$$

Here I set both $\alpha = \beta = 0.5$. Moreover, to further encourage thin vessel learning, I applied a class weight of [1.0, 1.0, 3.0] across the 3 output classes (background, regular vessels, and thin vessels), which gives more importance to small vessel regions that are harder to predict

Step 4: Optimization

For optimization, I selected AdamW because it is an improved version of the traditional Adam optimizer. AdamW is said to provide better regularization and generalization compared to Adam.

Additionally, I employed Cosine Annealing learning rate scheduler to dynamically adjust the learning rate during training. This scheduling method gradually reduces the learning rate following a cosine function, which allows faster convergence at the beginning and more stable updates toward the end of training.

The initial learning rate was set to 1e-3, with weight decay of 1e-2. The scheduler gradually decreased the learning rate to 1e-6 over every 100 epoch before restarting the cycle. This cycle pattern is believed to allow the model to escape the local minima and achieve better generalization performance.

Step 5: Model selection

Model	Normalization		Without Normalization	
	Dice	IoU	Dice	IoU
UNet 2 blocks	0.791	0.655	0.702	0.543
UNet 3 blocks	0.825	0.703	0.714	0.558
UNet 4 blocks	0.823	0.699		
ResUNet 3 blocks	0.820	0.695	0.681	0.518
ResUNet 4 blocks	0.823	0.700		

Table 1. Model performance on test set. BatchNorm was used as normalization

Model	Number of Parameters
UNet 2 blocks	1.87M
UNet 3 blocks	7.19M
UNet 4 blocks	31.04M
ResUNet 3 blocks	8.05M
ResUNet 4 blocks	32.45M

Table 2. Number of trainable parameters for each model

To overcome overfit, I applied extensive data augmentation and randomly cropping into small patches to help the model see more diverse samples and prevent it from memorizing specific image patterns.

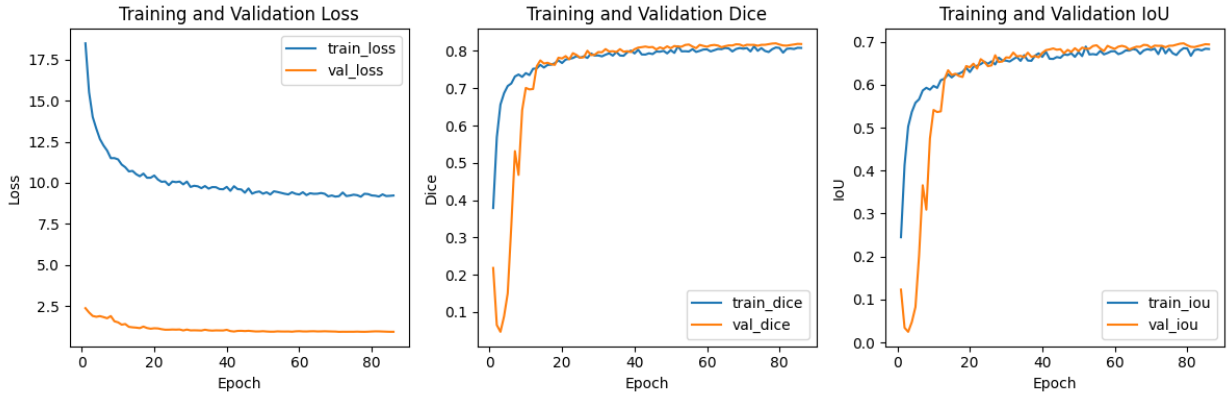
Additionally, at the bottleneck of UNet, I added a dropout layer with the rate of 0.5. The intuition is to encourage the model to work hard on the more abstract features. Early stopping is also used to prevent overfitting.

To avoid underfitting, I experimented with deeper UNet architectures to ensure the model had sufficient capacity to capture complex vessel structures.

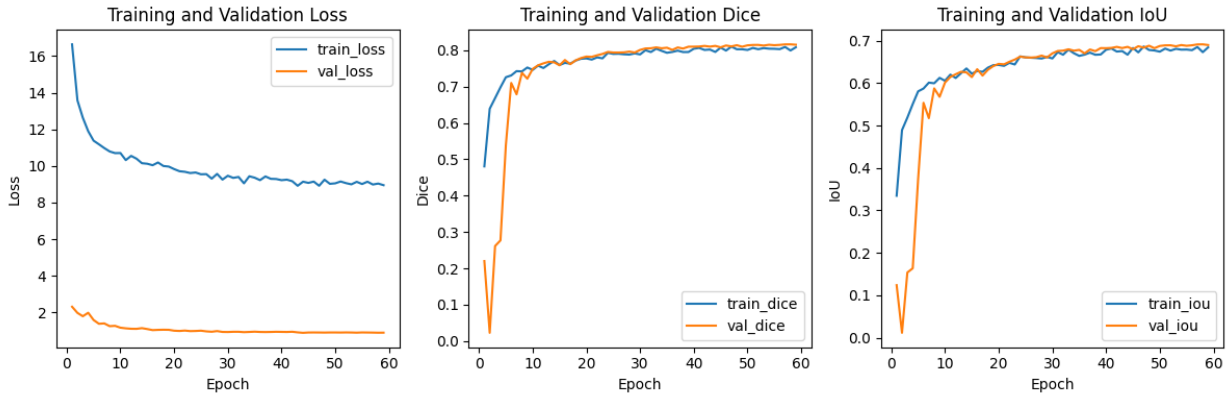
Step 6: Model demo

A. This section shows the training plots including training and validation loss, Dice, and IoU of each best and worst model:

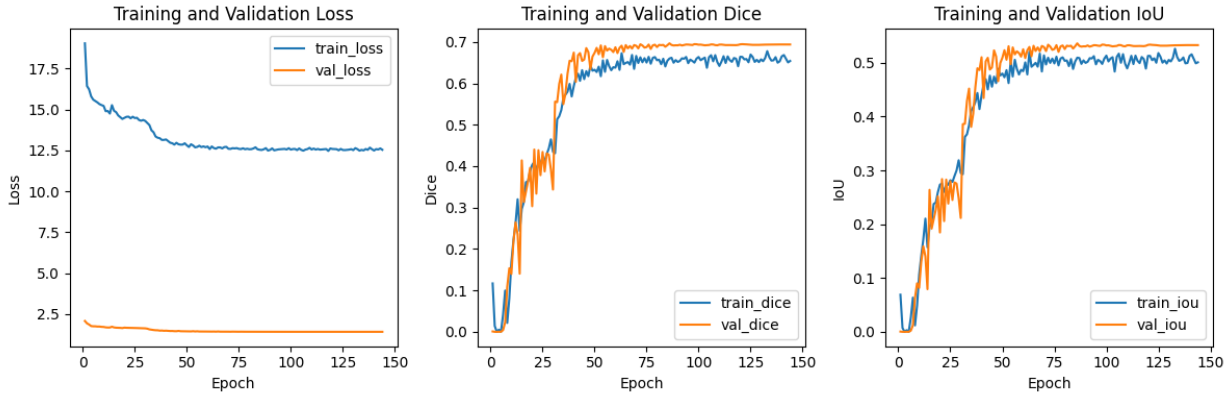
- UNet 3 blocks (Best):



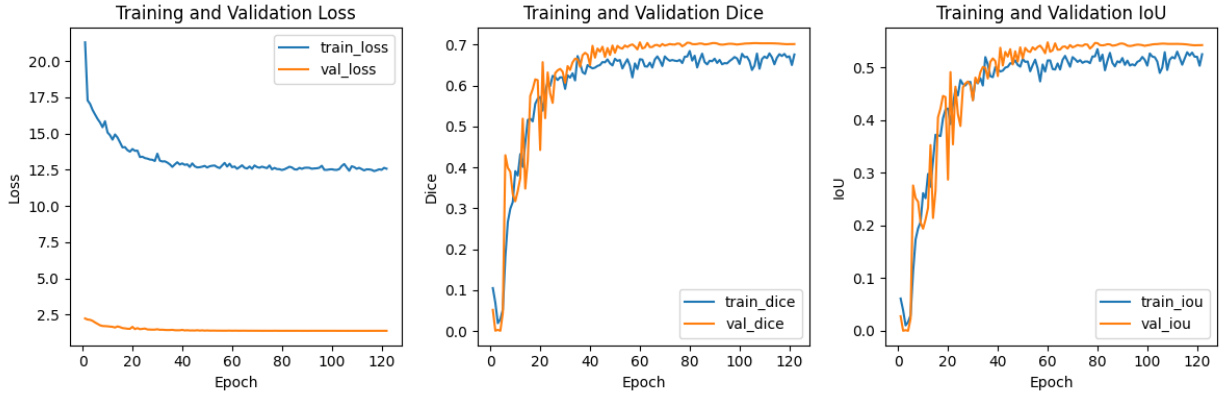
- ResUNet 4 blocks:



- ResUNet 3 blocks without normalization (Worst):



- UNet 2 blocks without normalization:

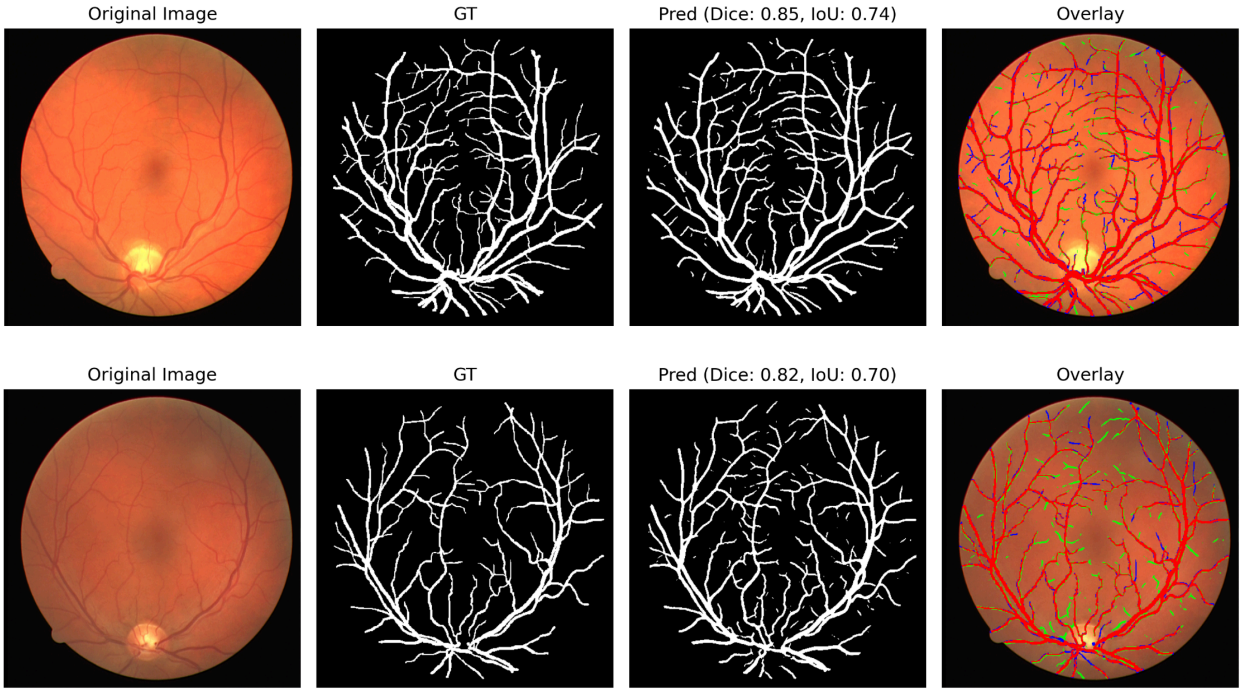


B. This section shows the predictions of each best and worst model:

To clarify the notation, in the overlay plot:

- Red: both ground truth and prediction (True Positives)
- Green: only prediction (False Positives)
- Blue: only ground truth (False Negatives)

1. UNet 3 blocks (Best):



It's interesting to notice that the model is able to predict the hard-to-see small vessels that were not annotated in the ground truth sometime:

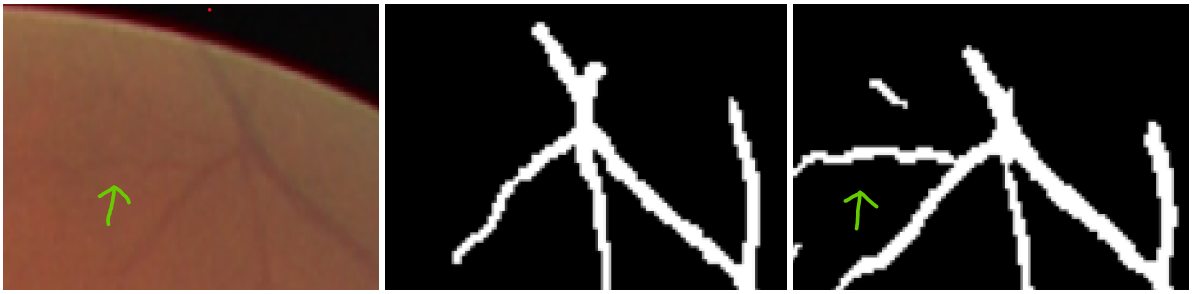
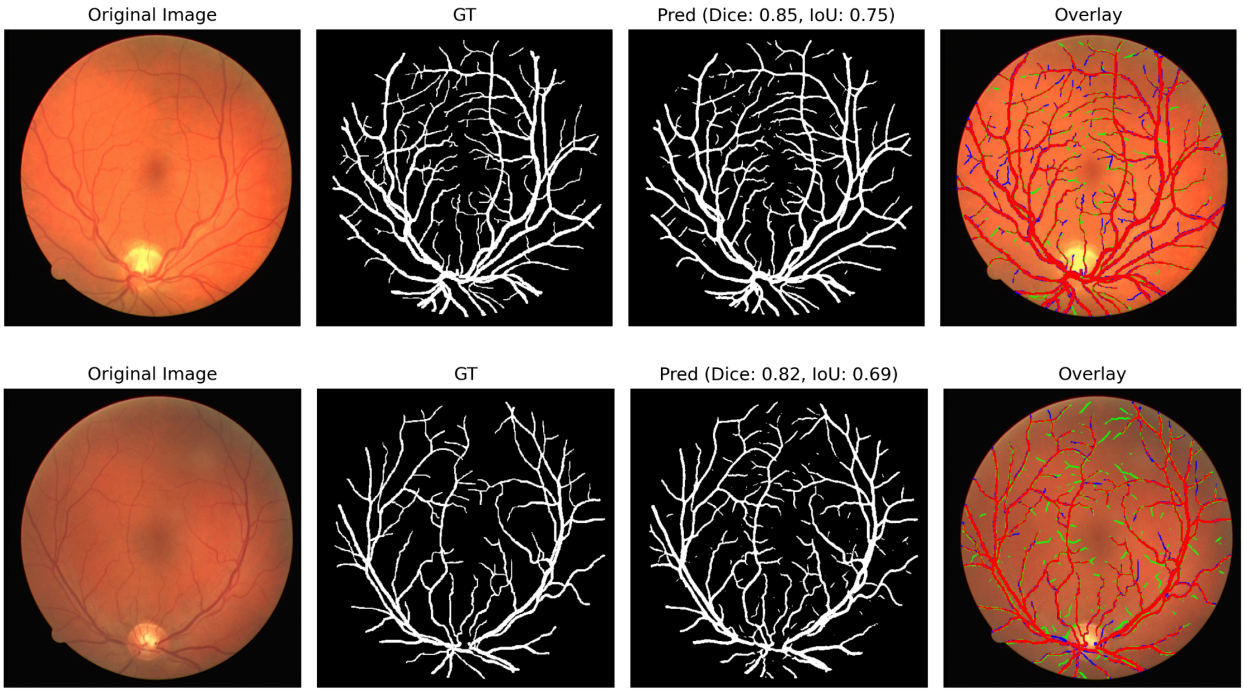
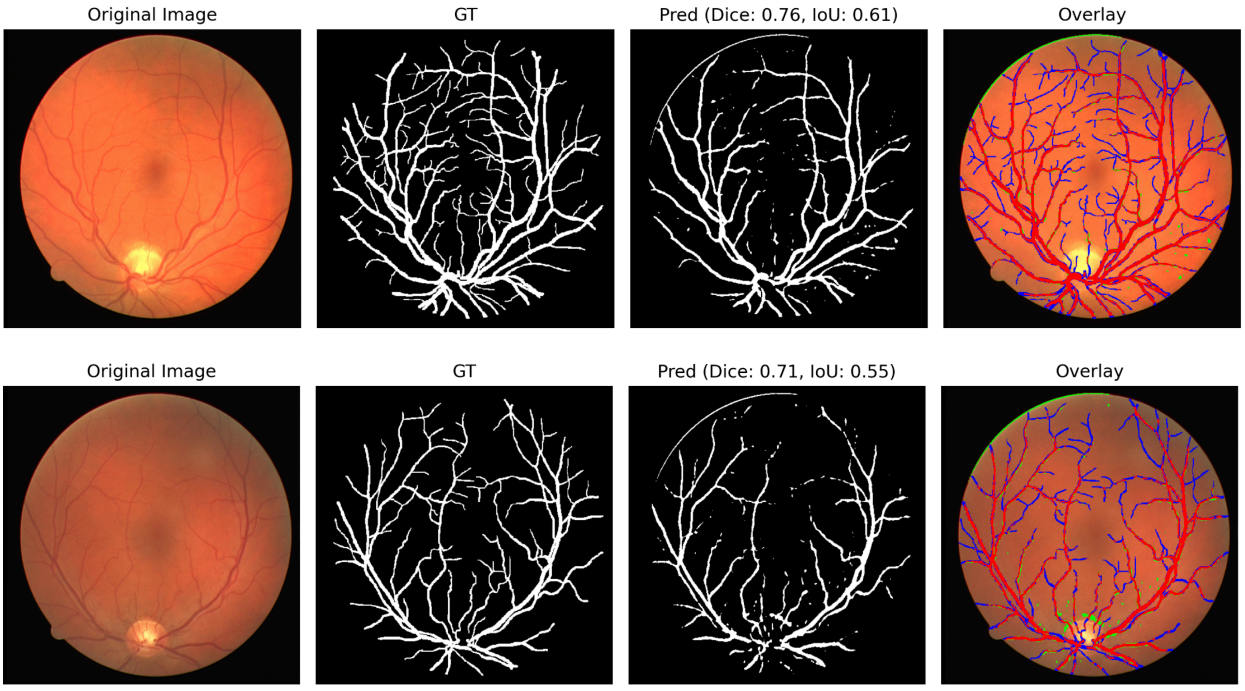


Figure 2. Left: original image, Middle: ground truth mask, Right: prediction mask

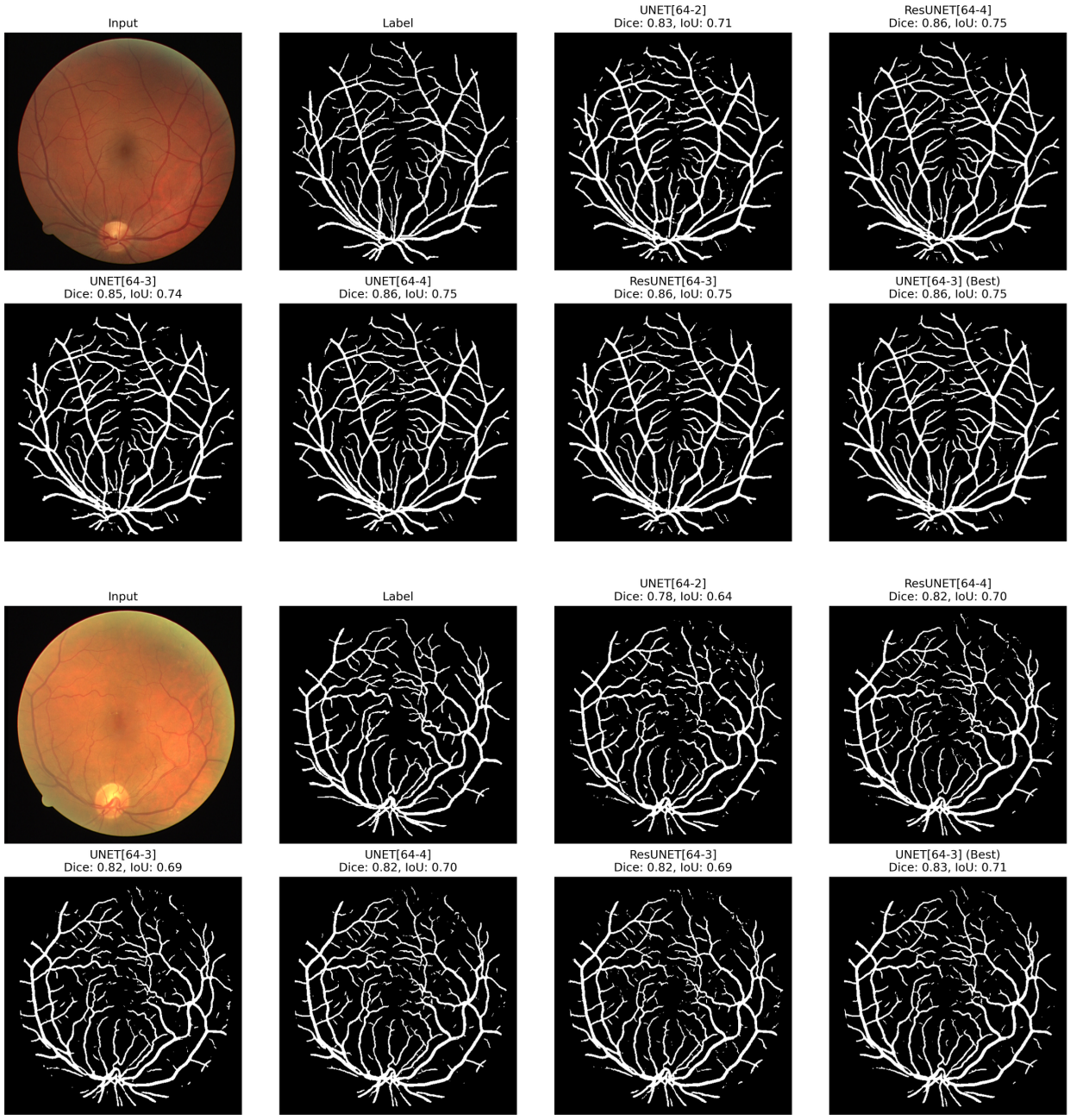
2. ResUNet 4 blocks:

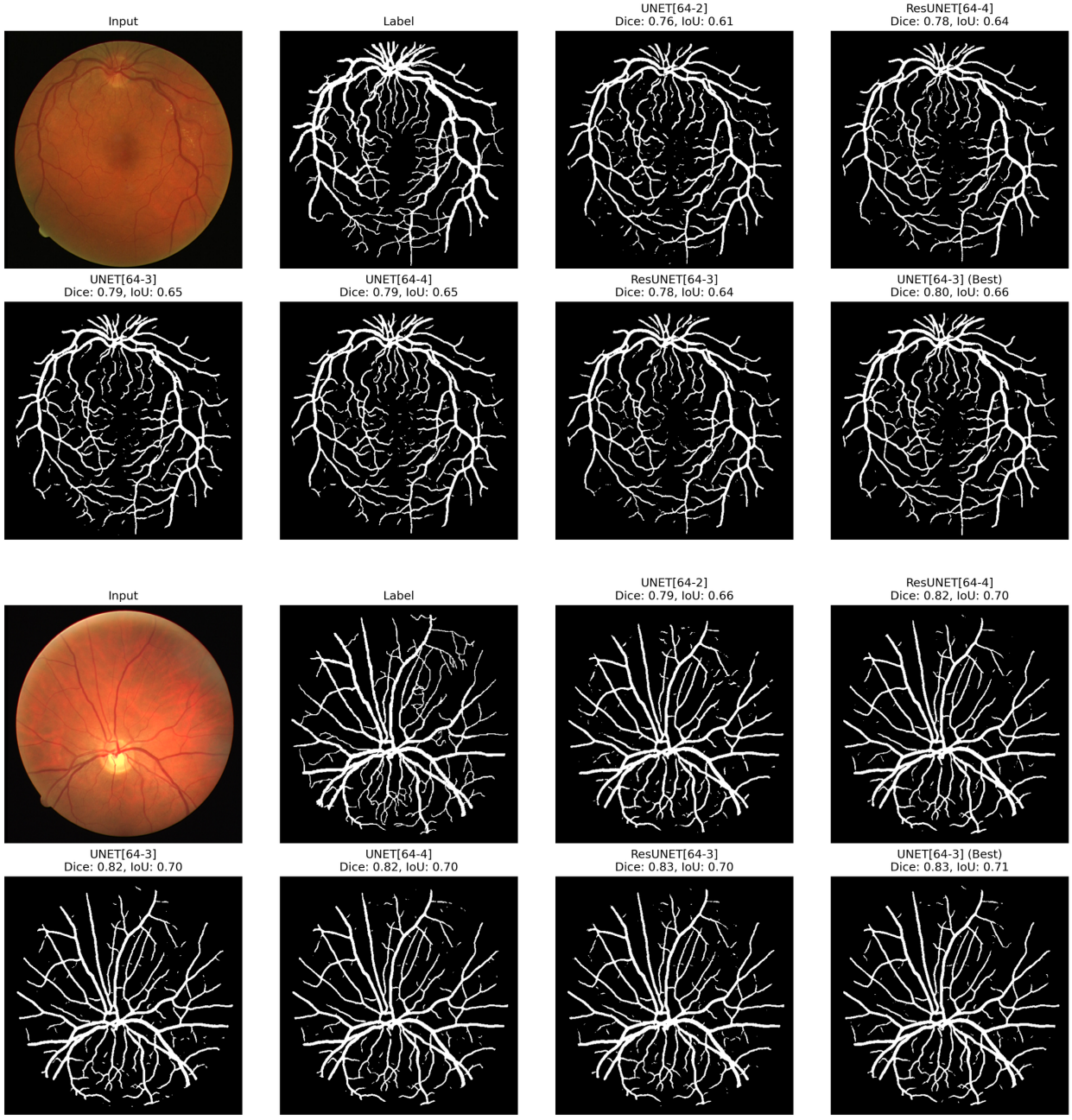


3. ResUNet 3 blocks without normalization (Worst):



C. This section shows the comparisons between each model architecture:




Input
Label
UNET[64-2]
Dice: 0.76, IoU: 0.61
ResUNET[64-4]
Dice: 0.78, IoU: 0.64
UNET[64-3]
Dice: 0.79, IoU: 0.65
UNET[64-4]
Dice: 0.79, IoU: 0.65
ResUNET[64-3]
Dice: 0.78, IoU: 0.64
UNET[64-3] (Best)
Dice: 0.80, IoU: 0.66
Input
Label
UNET[64-2]
Dice: 0.79, IoU: 0.66
ResUNET[64-4]
Dice: 0.82, IoU: 0.70
UNET[64-3]
Dice: 0.82, IoU: 0.70
UNET[64-4]
Dice: 0.82, IoU: 0.70
ResUNET[64-3]
Dice: 0.83, IoU: 0.70
UNET[64-3] (Best)
Dice: 0.83, IoU: 0.71